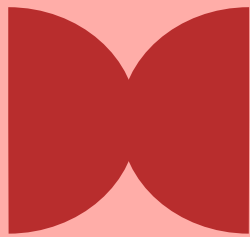


CSCI/MATH  
608:  
Data  
Science for  
Graduate  
Students



Dr. Kathleen Isenegger  
(she/her)

Week 4–Wednesday

# Cleaning and Wrangling Data

- ❑ Python Data Types
- ❑ Python Data Structures
- ❑ Tidying Data
- ❑ More on Filtering Rows with [ ]
- ❑ More on loc[]
- ❑ Extracting rows and columns by position with iloc[]
- ❑ Summary Statistics
- ❑ Applying functions to multiple columns
- ❑ Combine DataFrames with merge

# Learning Goals

- ❑ Define the term “tidy data”.
- ❑ Discuss the advantages of storing data in a tidy data format.
- ❑ Define what series and data frames are in Python, and describe how they relate to each other.
- ❑ Describe the common types of data in Python and their uses.

# Learning Goals

- ❑ Use the following functions for their intended data wrangling tasks:
  - ❑ melt
  - ❑ pivot
  - ❑ reset\_index
  - ❑ str.split
  - ❑ agg
  - ❑ assign and regular column assignment
  - ❑ groupby
  - ❑ merge

# Learning Goals

- ❑ Use the following operators for their intended data wrangling tasks:
  - ❑ `==`, `!=`, `<`, `>`, `<=`, and `>=`
  - ❑ `isin`
  - ❑ `&` and `|`
  - ❑ `[]`, `loc[]`, and `iloc[]`

# Python Data Types

| Data type             | Abbreviation          | Description                          | Example                    |
|-----------------------|-----------------------|--------------------------------------|----------------------------|
| integer               | <code>int</code>      | positive/negative/zero whole numbers | <code>42</code>            |
| floating point number | <code>float</code>    | real number in decimal form          | <code>3.14159</code>       |
| boolean               | <code>bool</code>     | true or false                        | <code>True</code>          |
| string                | <code>str</code>      | text                                 | <code>"Hello World"</code> |
| none                  | <code>NoneType</code> | represents no value                  | <code>None</code>          |

# Python Data Structures

| Data Structure | Description                                                                                    |
|----------------|------------------------------------------------------------------------------------------------|
| list           | An ordered collection of values that can store multiple data types at once.                    |
| dict           | A labeled data structure where <b>keys</b> are paired with <b>values</b>                       |
| Series         | An ordered collection of values <i>with labels</i> that can store multiple data types at once. |
| DataFrame      | A labeled data structure with <b>Series</b> columns of potentially different types.            |

# Lists

- ❑ Ordered collections of values
- ❑ Can contain any data type
- ❑ Elements go between `[]` with commas separating

```
cities = ["Toronto", "Vancouver", "Montreal", "Calgary", "Ottawa", "Winnipeg"]
```

# Series

- ❑ **Series** to represent DataFrame columns
- ❑ Contain one or more elements (like a list)
- ❑ Can contain a mix of data types, but good practice to include only one type in a Series
- ❑ The **pd.Series()** function creates a new Series from a list

```
region = pd.Series(["Toronto", "Montreal", "Vancouver", "Calgary", "Ottawa"])
```

```
cities_series = pd.Series(cities)
```

# DataFrames and Series

- ❑ A DataFrame is a collection of Series that are all of the same length
- ❑ Both DataFrames and Series are **data structures**
- ❑ Series are just really simple DataFrames

| Series of type character | Series of type integer | Series of type logical |
|--------------------------|------------------------|------------------------|
| region                   | year                   | voted                  |
| Toronto                  | 2016                   | TRUE                   |
| Vancouver                | 2016                   | TRUE                   |
| Montreal                 | 2016                   | TRUE                   |
| Calgary                  | 2016                   | TRUE                   |
| Ottawa                   | 2016                   | TRUE                   |
| Winnipeg                 | 2016                   | TRUE                   |
| Hamilton                 | 2016                   | TRUE                   |
| Edmonton                 | 2016                   | TRUE                   |
| Halifax                  | 2016                   | TRUE                   |

# Revisiting Selecting Columns with []

- ❑ Recall that we can make a new DataFrame with columns from an existing one

```
small_lang = tidy_lang[["language", "region", "most_at_home"]]
```

- ❑ Series can be made similarly, but with a single string (one column name) instead of a list

```
small_lang = tidy_lang["language"]
```

# Dictionaries

- ❑ A collection of key and value pairs
- ❑ Keys are usually integers or strings
- ❑ Values can be any data type
- ❑ Go between { } (curly braces)
- ❑ Key first, then value, separated by a :
- ❑ Commas separate dictionary entries

```
population_in_2016 = {  
    "cities": ["Toronto", "Vancouver", "Montreal", "Calgary", "Ottawa",  
    "Winnipeg"],  
    "population": [2235145, 1027613, 1823281, 544870, 571146, 321484]  
}
```

# Dictionaries and DataFrames

- ❑ Dictionaries can be converted to DataFrames
  - ❑ Keys become column names
  - ❑ Values become entries for those columns

```
population_in_2016_df = pd.DataFrame(population_in_2016)
```

```
population_in_2016_df = pd.DataFrame({  
    "cities": ["Toronto", "Vancouver", "Montreal", "Calgary",  
    "Ottawa", "Winnipeg"],  
    "population": [2235145, 1027613, 1823281, 544870, 571146,  
    321484]  
})
```

# Tidy Data

- ❑ A **tidy DataFrame** satisfies three criteria
  - ❑ each row is a single observation
  - ❑ each column is a single variable
  - ❑ each value is a single cell
- ❑ Why should data be tidy?
  - ❑ So it works with Pandas functions
  - ❑ Easier for humans to interpret

# Going from wide to long using melt

- For when data that should all be in one column is split over multiple columns

The diagram illustrates the transformation of a wide table into a long table using the melt function. The wide table on the left has columns for region and three years (2006, 2011, 2016). The long table on the right has columns for region, year, and population. The transformation is shown by arrows: a blue arrow points from the year columns of the wide table to the year column of the long table, and an orange arrow points from the region column of the wide table to the region column of the long table.

| region    | 2006     | 2011     | 2016    |
|-----------|----------|----------|---------|
| Toronto   | 31612897 | 33476688 | 5928040 |
| Vancouver | 2116581  | 2313328  | 2463431 |
| Montreal  | 3635556  | 3824221  | 4098927 |
| Calgary   | 1079310  | 1214839  | 1392609 |
| Ottawa    | 1133633  | 1236324  | 1323783 |

| region    | year | population |
|-----------|------|------------|
| Toronto   | 2006 | 31612897   |
| Vancouver | 2006 | 2116581    |
| Montreal  | 2006 | 3635556    |
| Calgary   | 2006 | 1079310    |
| Ottawa    | 2006 | 1133633    |
| Toronto   | 2011 | 33476688   |
| Vancouver | 2011 | 2313328    |
| Montreal  | 2011 | 3824221    |
| Calgary   | 2011 | 1214839    |
| Ottawa    | 2011 | 1236324    |
| Toronto   | 2016 | 5928040    |
| Vancouver | 2016 | 2463431    |
| Montreal  | 2016 | 4098927    |
| Calgary   | 2016 | 1392609    |
| Ottawa    | 2016 | 1323783    |

# Going from wide to long using melt

| category                      | language          | Toronto | Montreal | Vancouver | Calgary | Edmonton |
|-------------------------------|-------------------|---------|----------|-----------|---------|----------|
| Aboriginal                    | Aboriginal n.o.s. | 80      | 30       | 70        | 20      | 25       |
| Non-Official & Non-Aboriginal | Afrikaans         | 985     | 90       | 1435      | 960     | 575      |

| category                      | language          | region    | mother_tongue |
|-------------------------------|-------------------|-----------|---------------|
| Aboriginal                    | Aboriginal n.o.s. | Toronto   | 80            |
| Aboriginal                    | Aboriginal n.o.s. | Montreal  | 30            |
| Aboriginal                    | Aboriginal n.o.s. | Vancouver | 70            |
| Aboriginal                    | Aboriginal n.o.s. | Calgary   | 20            |
| Aboriginal                    | Aboriginal n.o.s. | Edmonton  | 25            |
| Non-Official & Non-Aboriginal | Afrikaans         | Toronto   | 985           |
| Non-Official & Non-Aboriginal | Afrikaans         | Montreal  | 90            |
| Non-Official & Non-Aboriginal | Afrikaans         | Vancouver | 1435          |
| Non-Official & Non-Aboriginal | Afrikaans         | Calgary   | 960           |
| Non-Official & Non-Aboriginal | Afrikaans         | Edmonton  | 575           |

```
lang_wide.melt(  
    id_vars=["category", "language"],  
    var_name="region",  
    value_name="mother_tongue",  
)
```

- 1 data frame object we want to reshape
- 2 columns to be used as identifier variables
- 3 name of the new column to be created whose values will come from the **names of the columns** that we want to combine
- 4 name of the new column to be created whose values will come from the **values of the columns** we want to combine

# Going from wide to long using melt

```
lang_mother_tidy = lang_wide.melt(  
  id_vars=["category", "language"],  
  var_name="region",  
  value_name="mother_tongue",  
)
```

# Going from long to wide using pivot

- For when data pertaining to one observation is split across multiple rows

| region    | year | type         | count   |
|-----------|------|--------------|---------|
| Toronto   | 2016 | population   | 5928040 |
| Toronto   | 2016 | commuters    | 2566700 |
| Toronto   | 2016 | incorporated | 1834    |
| Vancouver | 2016 | population   | 2463431 |
| Vancouver | 2016 | commuters    | 1006600 |
| Vancouver | 2016 | incorporated | 1886    |
| Montreal  | 2016 | population   | 4098927 |
| Montreal  | 2016 | commuters    | 1757100 |
| Montreal  | 2016 | incorporated | 1832    |
| Calgary   | 2016 | population   | 1392609 |
| Calgary   | 2016 | commuters    | 587300  |
| Calgary   | 2016 | incorporated | 1884    |
| Ottawa    | 2016 | population   | 1323783 |
| Ottawa    | 2016 | commuters    | 595900  |
| Ottawa    | 2016 | incorporated | 1855    |

| region    | year | population | commuters | incorporated |
|-----------|------|------------|-----------|--------------|
| Toronto   | 2016 | 5928040    | 2566700   | 1834         |
| Vancouver | 2016 | 2463431    | 1006600   | 1886         |
| Montreal  | 2016 | 4098927    | 1757100   | 1832         |
| Calgary   | 2016 | 1392609    | 587300    | 1884         |
| Ottawa    | 2016 | 1323783    | 595900    | 1855         |

| region   | category   | language          | type         | count |
|----------|------------|-------------------|--------------|-------|
| Montreal | Aboriginal | Aboriginal n.o.s. | most_at_home | 15    |
| Montreal | Aboriginal | Aboriginal n.o.s. | most_at_work | 0     |
| Toronto  | Aboriginal | Aboriginal n.o.s. | most_at_home | 50    |
| Toronto  | Aboriginal | Aboriginal n.o.s. | most_at_work | 0     |
| Calgary  | Aboriginal | Aboriginal n.o.s. | most_at_home | 5     |
| Calgary  | Aboriginal | Aboriginal n.o.s. | most_at_work | 0     |

| region   | category   | language          | most_at_home | most_at_work |
|----------|------------|-------------------|--------------|--------------|
| Montreal | Aboriginal | Aboriginal n.o.s. | 15           | 0            |
| Toronto  | Aboriginal | Aboriginal n.o.s. | 50           | 0            |
| Calgary  | Aboriginal | Aboriginal n.o.s. | 5            | 0            |

Going from  
long to wide  
using pivot

```
lang_long.pivot(  
    index=["region", "category", "language"],  
    columns=["type"],  
    values=["count"]  
) .reset_index()
```

- 1 data frame object we want to reshape
- 2 columns to use to make the new data frame's index
- 3 name of the column(s) from which to take the variable names
- 4 name of the column from which to take the values
- 5 reset the indexing

# Going from long to wide using pivot

```
lang_home_tidy = lang_long.pivot(  
    index=["region", "category", "language"],  
    columns=["type"],  
    values=["count"]  
) .reset_index()
```

# Separating strings into multiple cells with `str.split`

- ❑ For when the cells in a column contain string data that should be in two columns

The diagram illustrates the components of the pandas `str.split` method. It features a central code snippet: `lang_messy_longer["value"].str.split("/", expand=True)`. Four blue arrows point from descriptive text labels to specific parts of the code: 'data frame' points to `lang_messy_longer`, 'separator on which to split' points to the forward slash `/`, 'the column to split' points to the string `"value"`, and 'expand split strings into separate columns' points to the `expand=True` argument.

```
lang_messy_longer["value"].str.split("/", expand=True)
```

data frame

separator on which to split

the column to split

expand split strings into separate columns

# Separating strings into multiple cells with str.split

```
tidy_lang[["most_at_home", "most_at_work"]] = lang_messy_longer["value"].str.split("/", expand=True)
```

# More on Filtering Rows with [ ]

- ❑ Recall we can create a DataFrame with a subset of rows from an existing DataFrame by using a logical statement:

|                                 |                                 |
|---------------------------------|---------------------------------|
| <code>column == value</code>    | <code>column != value</code>    |
| <code>column &lt; value</code>  | <code>column &gt; value</code>  |
| <code>column &lt;= value</code> | <code>column &gt;= value</code> |

# More on Filtering Rows with [ ]

- ❏ We can filter on two or more conditions using &

```
french_mont = tidy_lang[  
  (tidy_lang["region"] == "Montréal") &  
  (tidy_lang["language"] == "French")  
]
```

# More on Filtering Rows with [ ]

- ❑ We can filter for rows that fulfill one of multiple conditions using | (or)

```
cal_ed =  
official_langs[  
    (official_langs["region"] == "Calgary") |  
    (official_langs["region"] == "Edmonton")  
]
```

# More on Filtering Rows with [ ]

- ❑ The `isin` function is used to filter rows from a list of values

```
city_names = ["Toronto", "Montréal", "Vancouver", "Calgary",  
              "Edmonton"]  
five_cities = region_data[region_data["region"].isin(city_names)]
```

# More on Filtering Rows and Selecting Columns with loc[]

- ❏ Recall if we want to filter and select in one step, we use loc[]

```
small_lang = tidy_lang.loc[  
    tidy_lang["region"] == "Toronto",  
    ["language", "region", "most_at_home", "most_at_work"]  
]
```

# More on Filtering Rows and Selecting Columns with loc[]

- ❏ We can also use loc[] to select *ranges* of columns using :

```
small_lang = tidy_lang.loc[  
    tidy_lang["region"] == "Toronto",  
    "language":"most_at_work"  
]
```

# More on Filtering Rows and Selecting Columns with loc[]

- ❑ Passing : in place of a logical statement means to include all rows

```
small_lang = tidy_lang.loc[:, "language": "most_at_work"]
```

- ❑ Omitting a column name on one side of the : means either "everything before" or "everything after" the column

```
after_lang = tidy_lang.loc[:, "language":]  
up_to_lang = tidy_lang.loc[:, : "language"]
```

# More on Filtering Rows and Selecting Columns with loc[]

- ❏ We can select columns with logical statements using loc[]

```
start_most = tidy_lang.loc[:, tidy_lang.columns.str.startswith("most")]
contains_ = tidy_lang.loc[:, tidy_lang.columns.str.contains("_")]
```

# Extracting rows and columns by position with `iloc[]`

- ❑ `iloc[]` provides the ability to index with column/row positions
- ❑ You can use `:` for ranges

```
small_lang = tidy_lang.iloc[10:20, 2]
```

# Summary Statistics for One Column

- ❑ We can use several functions for summary statistics
  - ❑ count, mean, median, std, max, min, sum

```
lang_max = region_lang["most_at_home"].FUNCTIONNAME()
```

- ❑ We can compute multiple statistics with **agg**

```
min_max = region_lang["most_at_home"].agg(["min", "max"])
```

- ❑ Or just get a full summary with **describe**

```
stats = region_lang["most_at_home"].describe()
```

# Summary Statistics on DataFrames

- ❑ Using summary statistics columns on a DataFrame returns the value for all columns
- ❑ For statistics that need numbers, specify **numeric\_only=True**

```
means = region_lang.mean(numeric_only=True)
```

# Summary Statistics on Groups

- ❑ **groupby** is used to divide the observations of a DataFrame into groups based on some criteria
- ❑ We can then apply a summary statistic function (or transformation, or filter) to each group
- ❑ We use **value\_counts** to get the number of observations (or proportion) per group

```
group_count = region_lang.value_counts("region")  
group_prop = region_lang.value_counts("region", normalize =True)
```

# Summary Statistics on Groups

- Imagine I want to find the max values for **most\_at\_home** for each region in Canada

|              | region   | category             | language                     | most_at_home | most_at_work | lang_known |
|--------------|----------|----------------------|------------------------------|--------------|--------------|------------|
| <sup>0</sup> | Toronto  | Aboriginal languages | Aboriginal languages, n.o.s. | 50           | 0            | 0          |
| <sup>1</sup> | Edmonton | Aboriginal languages | Yiddish                      | 0            | 0            | 0          |
| <sup>2</sup> | Toronto  | Aboriginal languages | Afrikaans                    | 265          | 0            | 0          |

# Summary Statistics on Groups

- ❑ I need to group by the region and then call the max function

```
maximums = region_lang.groupby("region")["most_at_home"].max().reset_index()
```

- ❑ You can also group by multiple columns at once

```
maximums = region_lang.groupby(["region", "category"])["most_at_home"].max().reset_index()
```

# Applying Functions to Multiple Columns

- ❑ **astype** changes the type of the data within a column

```
region_lang["mother_tongue"] = region_lang["mother_tongue"].astype("int32")
```

- ❑ We can also apply **row-wise** functions to produce a new column that summarizes each row with

```
region_lang_nums["max_value"] = region_lang_nums.max(axis=1)
```

# Applying Functions to Multiple Columns

- ❑ We can also use **assign** to make a whole new DataFrame

```
region_lang_big = region_lang.assign(maximum=region_lang_nums.max(axis=1))
```

- ❑ **div** allows us to divide by the values in a column

```
english_lang.loc[:, "mother_tongue": "lang_known"] =  
english_lang.loc[:,  
    "mother_tongue": "lang_known"  
].div(english_lang["city_pops"], axis=0)
```

# Combine DataFrames with merge

- ❑ If we have two DataFrames that share a column, we can combine them with **merge**

```
english_city = english_lang.merge(city_populations, on="region")
```

# Homework Reminder



Big timers



MINS



MINS



MINS

Big timers

